ChatGPT said:

C++17 Enterprise Backend Stack: Database + Redis

You can build the same enterprise projects in **C++17 instead of Java**: banking systems, inventory management, e-commerce backends, authentication services, order processing, and job schedulers.

Since your focus is on **C++17, REST APIs, server-side development, and no embedded development**, I recommend a stack built around a C++ web framework, PostgreSQL, Redis, and NGINX.

1. Recommended C++17 Stack

Layer	Java Equivalent	C++17 Technology
Programming language	Java	**C++17**
Backend framework	Spring Boot	**Drogon or POCO**
HTTP REST API	Spring Web MVC	Drogon HTTP controllers or POCO HTTP server
Database	PostgreSQL	**PostgreSQL**
Database access	Spring Data JPA / Hibernate	Drogon ORM or POCO Data
Cache	Redis	**Redis**
Redis client	Spring Data Redis	redis-plus-plus or hiredis
Authentication	Spring Security	JWT library + framework middleware
JSON processing	Jackson	nlohmann/json
API testing	Postman	Postman
API documentation	OpenAPI / Swagger	OpenAPI integration
Logging	SLF4J / Logback	spdlog
Unit testing	JUnit / Mockito	GoogleTest

Layer	Java Equivalent	C++17 Technology
Build system	Maven / Gradle	**CMake**
Reverse proxy	NGINX	NGINX
Containerization	Docker	Docker
Cloud deployment	AWS	AWS
CI/CD	GitHub Actions / Jenkins	GitHub Actions / Jenkins

My preferred starting stack: C++17 + Drogon + PostgreSQL + Redis + NGINX + CMake + Docker.

Drogon provides a more integrated web-development experience, including HTTP routing, asynchronous request handling, database integration, and ORM support. POCO is a reasonable alternative if you prefer assembling your backend from more focused C++ libraries.

Official references: Drogon documentation · POCO documentation · redis-plus-plus documentation.

2. Drogon vs POCO: Which Should You Choose?

Feature	Drogon	POCO
REST API development	Excellent	Good
C++17 support	Yes	Yes
PostgreSQL integration	Built-in ORM support	POCO Data PostgreSQL
Redis integration	Built-in support	POCO Redis
Asynchronous processing	Core design	Available through networking facilities
HTTP routing	Built in	Build with HTTP server components
Database object mapping	Built-in ORM	Data and ActiveRecord facilities
Best suited for	Integrated REST API applications	Modular networking and server applications

Both support the technologies needed for these projects. Drogon offers a more integrated starting point for a REST API with PostgreSQL and Redis. GitHub +2

My recommendation for you:

- Choose **Drogon** if you want to build complete enterprise REST API projects quickly.
- Choose **POCO** if you want to assemble the application from separate libraries and learn how the components fit together.

You have already explored both frameworks, so there is no need to learn another framework before building a complete project.

---

3. Example: C++17 Enterprise Application Architecture

Consider an Inventory Management System.

```
InventoryManagement/
│
├── src/
│   ├── main.cpp
│   │
│   ├── controllers/
│   │   └── ProductController.cpp
│   │
│   ├── services/
│   │   └── ProductService.cpp
│   │
│   ├── repositories/
│   │   └── ProductRepository.cpp
│   │
│   ├── models/
│   │   └── Product.cpp
│   │
│   ├── cache/
│   │   └── ProductCache.cpp
│   │
│   ├── security/
│   │   └── AuthMiddleware.cpp
│   │
│   └── common/
│       └── ErrorHandler.cpp
│
├── include/
├── tests/
├── migrations/
├── config/
├── CMakeLists.txt
├── Dockerfile
└── docker-compose.yml
```

How a request flows

Suppose Postman sends:

```
GET /api/products/101
```

The application processes it as follows:

1. **NGINX:** Receives the request and forwards it to the API server.
2. **Drogon controller:** Handles the HTTP request.
3. **ProductService:** Applies business rules.
4. **ProductCache:** Checks Redis for product `101` .
5. If the product is cached, return it.
6. Otherwise, **ProductRepository** fetches it from PostgreSQL.
7. Store the result in Redis with an expiration time.
8. Return the product as JSON.

```
Postman
   |
   v
NGINX
   |
   v
ProductController
   |
   v
ProductService
   |
   v
ProductCache -----> Redis
   |
   | Cache miss
   v
ProductRepository -> PostgreSQL
```

This is a common cache-aside pattern.

---

4. C++17 vs Java: Important Difference

Java's Spring ecosystem provides a relatively unified set of conventions for dependency injection, security, persistence, and configuration.

With C++17, you can build the same architectural layers, but the exact framework and library choices require more deliberate integration.

Concern	Java approach	C++17 approach
Business logic	Java classes	C++ classes
Dependency injection	Spring container	Constructor injection or a DI library
ORM	Hibernate	Drogon ORM or POCO ActiveRecord
Cache	Spring Data Redis	Framework Redis support or a Redis client
REST endpoints	Spring controllers	Drogon controllers or POCO HTTP handlers
Background workers	ExecutorService	`std::thread`, thread pools, or framework facilities
Configuration	Spring configuration	Framework configuration files
Build	Maven	CMake

You do **not** need to implement everything yourself. Use framework facilities where they help, while keeping business logic independent of HTTP, Redis, and database code where practical.

5. Final Stack I Would Use

For your first complete C++17 enterprise project:

```
Language       : C++17
Framework      : Drogon
REST API       : Drogon HTTP
Database       : PostgreSQL
ORM            : Drogon ORM
Cache          : Redis
JSON           : Drogon JSON support
Authentication : JWT + Drogon filters
Logging        : spdlog
Testing        : GoogleTest
Build          : CMake + Ninja
Proxy          : NGINX
Containers     : Docker
Cloud          : AWS EC2
Testing Client : Postman
```

Start with the Inventory Management System, then reuse the same architecture for banking, order management, and e-commerce projects.

One final point: C++17 is sufficient for this stack. You do not need C++20 or C++23 to build these projects.

If you want, I can:

- ↳ Add a JWT-based authentication example to the Inventory Management System
- ↳ Explain Redis cache eviction strategies in this C++17 backend context
- ↳ Show a CMakeLists.txt setup example for the Inventory Management System project